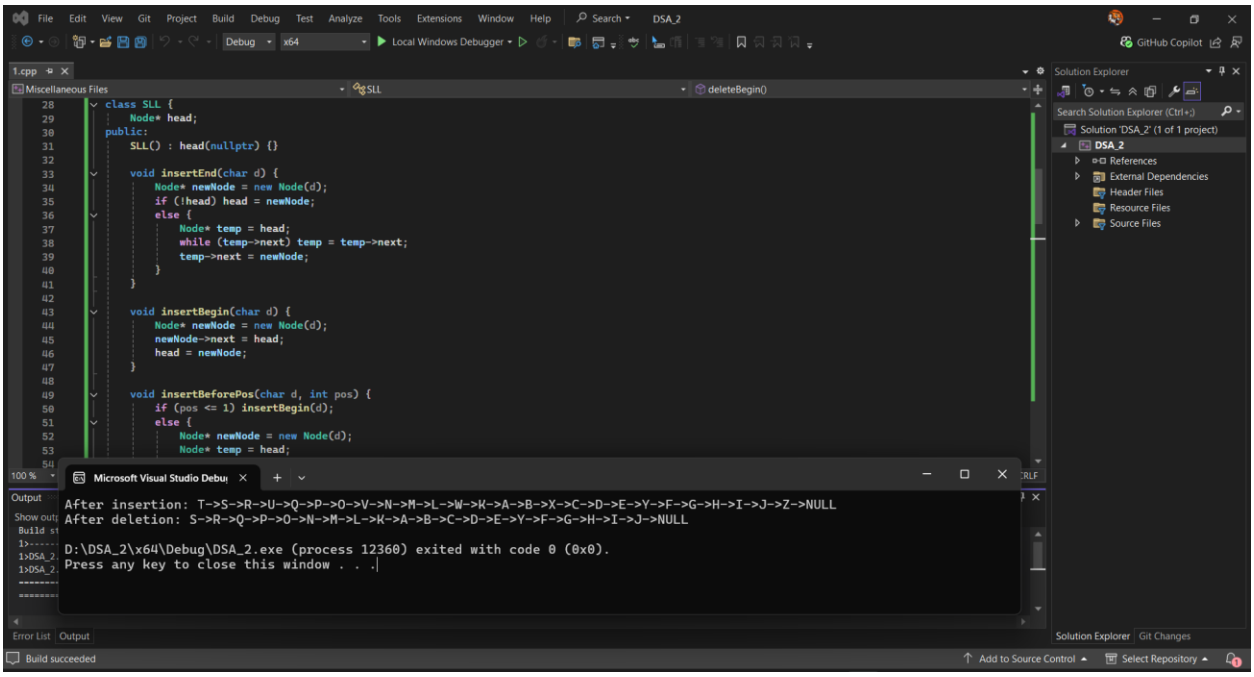
Name: Prashant Ranjan Roll No : 24MC3035

```
/*
Q.1: Given a series of 26 Alphabets- 'A', 'B', 'C', 'D', ......, 'Y', 'Z',
then write a program to perform the following operation in a Singly Linked
List (SLL).

Insertion:

(I) Perform the insertion of the first ten (10) alphabets into the SLL,
considering the insertion at the end of the list.
(II) Perform the insertion of the next ten (10) alphabet into the SLL,
considering the insertion at the beginning of the list.
(III) Perform the insertion of the remaining five (5) alphabets before the
4th, 8th, 12th, 16th, and 20th node of the linked list, and then insert
the last alphabet 'Z' at the end of the linked list.

Deletion:

(I) Perform the deletion of an alphabet from the beginning and another
from the end of the list.
(II) Delete the 3rd, 6th, 9th, and 12th alphabets/nodes from the linked
list

*/

// Solution

#include <iostream>
using namespace std;

struct Node {
    char data;
    Node* next;
    Node(char d) : data(d), next(nullptr) {}
};

class SLL {
    Node* head;
public:
    SLL() : head(nullptr) {}

    void insertEnd(char d) {
        Node* newNode = new Node(d);
        if (!head) head = newNode;
        else {
            Node* temp = head;
            while (temp->next) temp = temp->next;
            temp->next = newNode;
        }
    }

    void insertBegin(char d) {
```

```
        Node* newNode = new Node(d);
        newNode->next = head;
        head = newNode;
    }

    void insertBeforePos(char d, int pos) {
        if (pos <= 1) insertBegin(d);
        else {
            Node* newNode = new Node(d);
            Node* temp = head;
            for (int i = 1; i < pos - 1 && temp->next; i++) temp = temp-
>next;
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }

    void deleteBegin() {
        if (head) {
            Node* temp = head;
            head = head->next;
            delete temp;
        }
    }

    void deleteEnd() {
        if (!head) return;
        if (!head->next) {
            delete head;
            head = nullptr;
        }
        else {
            Node* temp = head;
            while (temp->next->next) temp = temp->next;
            delete temp->next;
            temp->next = nullptr;
        }
    }

    void deletePos(int pos) {
        if (!head || pos < 1) return;
        if (pos == 1) deleteBegin();
        else {
            Node* temp = head;
            for (int i = 1; i < pos - 1 && temp->next; i++) temp = temp-
>next;
            if (temp->next) {
                Node* toDelete = temp->next;
                temp->next = temp->next->next;
                delete toDelete;
            }
        }
    }
```

```
    void display() {
        Node* temp = head;
        while (temp) {
            cout << temp->data << "->";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
};

int main() {
    SLL list;

    // I: First 10 at end
    for (char c = 'A'; c <= 'J'; c++) list.insertEnd(c);

    // II: Next 10 at beginning
    for (char c = 'K'; c <= 'T'; c++) list.insertBegin(c);

    // III: Remaining 5 before positions
    char rem[] = { 'U','V','W','X','Y' };
    int pos[] = { 4,8,12,16,20 };
    for (int i = 0; i < 5; i++) list.insertBeforePos(rem[i], pos[i]);
    list.insertEnd('Z');

    cout << "After insertion: ";
    list.display();

    // I: Beginning and end
    list.deleteBegin();
    list.deleteEnd();

    // II: Specific positions
    int delPos[] = { 3,6,9,12 };
    for (int pos : delPos) list.deletePos(pos);

    cout << "After deletion: ";
    list.display();

    return 0;
}
```



// Output:

After insertion: T->S->R->U->Q->P->O->V->N->M->L->W->K->A->B->X->C->D->E->Y->F->G->H->I->J->Z->NULL

After deletion: S->R->Q->P->O->N->M->L->K->A->B->C->D->E->Y->F->G->H->I->J->NULL

D:\DSA_2\x64\Debug\DSA_2.exe (process 12836) exited with code 0 (0x0).

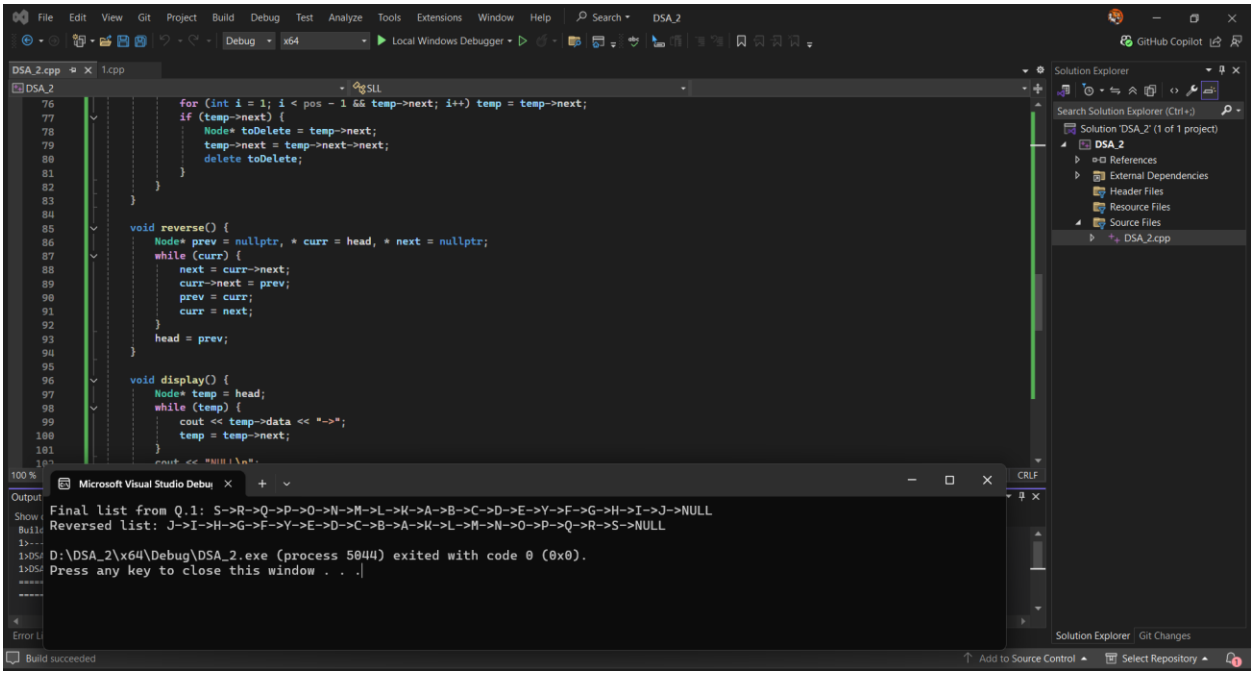
Press any key to close this window . . .

```
/*
Q.2: Write a program to reverse the singly linked list, and the input
linked list would be
the final output linked list after performing all operations in Q.1.
*/

// sol

#include <iostream>
using namespace std;

struct Node {
    char data;
    Node* next;
    Node(char d) : data(d), next(nullptr) {}
};

class SLL {
```

```
    Node* head;
public:
    SLL() : head(nullptr) {}

    void insertEnd(char d) {
        Node* newNode = new Node(d);
        if (!head) head = newNode;
        else {
            Node* temp = head;
            while (temp->next) temp = temp->next;
            temp->next = newNode;
        }
    }

    void insertBegin(char d) {
        Node* newNode = new Node(d);
        newNode->next = head;
        head = newNode;
    }

    void insertBeforePos(char d, int pos) {
        if (pos <= 1) insertBegin(d);
        else {
            Node* newNode = new Node(d);
            Node* temp = head;
            for (int i = 1; i < pos - 1 && temp->next; i++) temp = temp-
>next;
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }

    void deleteBegin() {
        if (head) {
            Node* temp = head;
            head = head->next;
            delete temp;
        }
    }

    void deleteEnd() {
        if (!head) return;
        if (!head->next) {
            delete head;
            head = nullptr;
        }
        else {
            Node* temp = head;
            while (temp->next->next) temp = temp->next;
            delete temp->next;
            temp->next = nullptr;
        }
    }
```

```
    void deletePos(int pos) {
        if (!head || pos < 1) return;
        if (pos == 1) deleteBegin();
        else {
            Node* temp = head;
            for (int i = 1; i < pos - 1 && temp->next; i++) temp = temp-
>next;
            if (temp->next) {
                Node* toDelete = temp->next;
                temp->next = temp->next->next;
                delete toDelete;
            }
        }
    }

    void reverse() {
        Node* prev = nullptr, * curr = head, * next = nullptr;
        while (curr) {
            next = curr->next;
            curr->next = prev;
            prev = curr;
            curr = next;
        }
        head = prev;
    }

    void display() {
        Node* temp = head;
        while (temp) {
            cout << temp->data << "->";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
};

int main() {
    SLL list;

    // Build the final linked list from Q.1 operations
    for (char c = 'A'; c <= 'J'; c++) list.insertEnd(c);
    for (char c = 'K'; c <= 'T'; c++) list.insertBegin(c);

    char rem[] = { 'U','V','W','X','Y' };
    int pos[] = { 4,8,12,16,20 };
    for (int i = 0; i < 5; i++) list.insertBeforePos(rem[i], pos[i]);
    list.insertEnd('Z');

    list.deleteBegin();
    list.deleteEnd();

    int delPos[] = { 3,6,9,12 };
    for (int p : delPos) list.deletePos(p);
```

```
    cout << "Final list from Q.1: ";
    list.display();

    list.reverse();
    cout << "Reversed list: ";
    list.display();

    return 0;
}
```



// Output:

Final list from Q.1: S->R->Q->P->O->N->M->L->K->A->B->C->D->E->Y->F->G->H->I->J->NULL

Reversed list: J->I->H->G->F->Y->E->D->C->B->A->K->L->M->N->O->P->Q->R->S->NULL

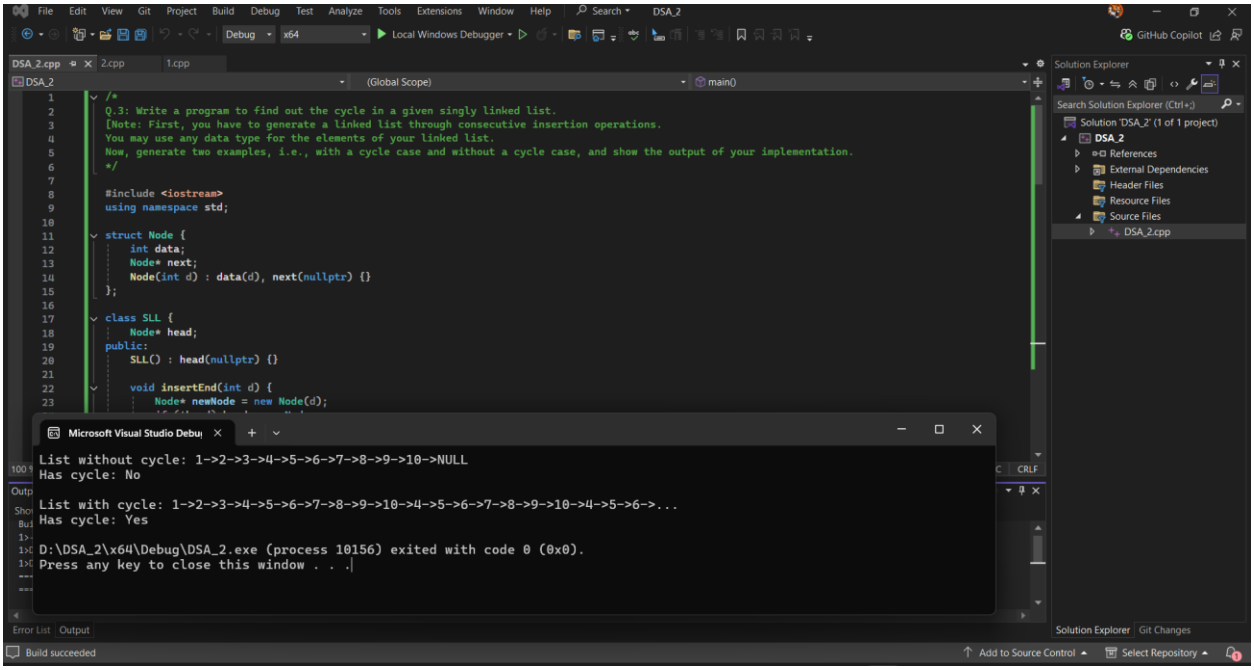
D:\DSA_2\x64\Debug\DSA_2.exe (process 8592) exited with code 0 (0x0).

Press any key to close this window . . .

```
/*
Q.3: Write a program to find out the cycle in a given singly linked list.
[Note: First, you have to generate a linked list through consecutive
insertion operations.
You may use any data type for the elements of your linked list.
Now, generate two examples, i.e., with a cycle case and without a cycle
case, and show the output of your implementation.
*/
```

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};

class SLL {
    Node* head;
public:
    SLL() : head(nullptr) {}

    void insertEnd(int d) {
        Node* newNode = new Node(d);
        if (!head) head = newNode;
        else {
            Node* temp = head;
            while (temp->next) temp = temp->next;
            temp->next = newNode;
        }
    }

    void createCycle(int pos) {
        if (!head) return;
        Node* end = head, * cycleNode = nullptr;
        int count = 1;

        while (end->next) {
            if (count == pos) cycleNode = end;
            end = end->next;
            count++;
        }
        if (cycleNode) end->next = cycleNode;
    }

    bool hasCycle() {
        if (!head) return false;

        Node* slow = head, * fast = head;
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast) return true;
        }
        return false;
    }

    void display(int limit = 20) {
        Node* temp = head;
        int count = 0;
        while (temp && count < limit) {
```

```
            cout << temp->data << "->";
            temp = temp->next;
            count++;
        }
        cout << (temp ? "..." : "NULL") << endl;
    }
};

int main() {
    SLL list1;
    for (int i = 1; i <= 10; i++) list1.insertEnd(i);

    cout << "List without cycle: ";
    list1.display();
    cout << "Has cycle: " << (list1.hasCycle() ? "Yes" : "No") << endl <<
endl;

    SLL list2;
    for (int i = 1; i <= 10; i++) list2.insertEnd(i);
    list2.createCycle(4);

    cout << "List with cycle: ";
    list2.display();
    cout << "Has cycle: " << (list2.hasCycle() ? "Yes" : "No") << endl;

    return 0;
}
```



// Output :

List without cycle: 1->2->3->4->5->6->7->8->9->10->NULL

Has cycle: No

List with cycle: 1->2->3->4->5->6->7->8->9->10->4->5->6->7->8->9->10->4->5->6->...

Has cycle: Yes

D:\DSA_2\x64\Debug\DSA_2.exe (process 10156) exited with code 0 (0x0).

Press any key to close this window . . .